Congestion-Tolerant and Auditable Trust Management for the Internet of Digital Twins

Abdulelah Alwabel

*Department of Computer Sciences, College of Computer Engineering and Sciences, Prince Sattam bin Abdulaziz University, Al-Kharj, Saudi Arabia.

Corresponding author(s). E-mail(s): a.alwabel@psau.edu.sa;

Abstract

The Internet of Digital Twins (IoDT) trust model pairs every IoT node with a cloud twin and isolates a node when any of four binary factors (energy, packet delivery ratio, traceability and twin notification) fails. This rule is fragile under congestion, trusts a single twin, and keeps verdicts in a mutable database. We propose three extensions: an exponentially smoothed weighted score with a per-factor persistence check; quorum voting over signed votes of replicated twins; and a signed Merkle audit log kept off the decision path. In simulation at 4% per-factor congestion, the smoothed score falsely isolated 1.1% of honest nodes within 16 rounds, against 93% for the binary AND and 14–26% for two-failure rules, at the cost of one extra round. On two public packet-delivery traces it kept the lowest false isolation everywhere, although bursty, concentrated real losses raised its rates to 2.0–31%. With identical evidence and two of five replicas corrupted, voting kept detection at the level of an honest twin, whereas one corrupted twin always hid its node. The audit log rejected every edit by adversaries holding fewer than a quorum of keys. Without congestion, an attacker that knows the rule can lie in 37.5% of rounds undetected.

Keywords: Audit log, Digital twin, Fog computing, Internet of Things, Quorum voting, Trust management

1 Introduction

The Internet of Things (IoT) is a network of interconnected devices in which trust is fundamental to the security, reliability and scalability of large deployments. Individual

devices are cheap, constrained and routinely exposed to physical and network compromise; a survey of IoT vulnerabilities, complemented by an empirical measurement of exploited IoT devices on the Internet, documents the practical consequences of this exposure [1]. Digital twin (DT) technology, reviewed for industry by Tao *et al.* [2] and surveyed for the IoT by Minerva, Lee and Crespi [3], mirrors the state of each device so that its activity can be cross-checked.

Soula *et al.* [4] turned this idea into the Internet of Digital Twins (IoDT) trust model. IoDT assigns one cloud-resident twin to every physical node and folds four binary factors — energy, packet delivery ratio (PDR), traceability and twin notification — into a single logical-AND verdict. In its iFogSim evaluation the model identifies the two compromised nodes in 92% and 95% of cases and isolates them within 250–300 ms. Three limitations remain.

1. *Fragility under transient congestion.* Under the AND, a single factor that drops for one round is enough to flag an honest node. The original evaluation already reports a false positive (node N3) and notes that legitimate nodes can be mistaken for compromised ones during collisions or high-traffic periods; its authors suggest adjusting the threshold, or adapting the PDR threshold under congestion, but do not evaluate such a variant [4]. Dynamic-trust schemes [5, 6] instead keep continuous scores that age out transient noise.
2. *A single, corruptible twin and an untested adaptive adversary.* The original evaluation considers attackers that send false notifications and a denial-of-service (DoS) attacker [4]. It does not consider on-off attackers that lie only occasionally to stay below the detection threshold [7], nor an adversary that corrupts the twin that watches a node, in which case nothing else in the pipeline can flag the node. Threats against the twin itself are part of the DT threat taxonomy of [8].
3. *No tamper-evident record.* Verdicts are stored in a central database that can be silently rewritten if the cloud is compromised. Blockchain-based IoT and fog frameworks provide tamper evidence [9–13], but most of them read from or write to the chain while a decision is being made, and the latency and computational overhead of blockchain validation are the reasons Soula *et al.* [4] give for avoiding blockchain in real-time IoT.

This paper keeps the IoDT factor set and addresses the three limitations with three extensions. Its contributions are:

- a statement of IoDT trust evaluation as a per-round classification problem, with metrics that separate per-round false positives, the operationally relevant *false-isolation* rate, the mean time to the first false isolation over long operation, and the cost of quarantine with re-admission (Section 3);
- Extension I, a weighted continuous score with exponential moving-average (EMA) smoothing and a per-factor persistence check. A closed form and a counting argument show that the binary AND errs at rate $\Theta(q)$ in the per-factor congestion probability q, any rule that needs two failures at $\Theta(q^2)$, and the smoothed score, at its default parameters and under independent congestion, at $\Theta(q^3)$. The evaluation therefore compares it with delay-matched baselines (a debounced AND and a 2-of-4 rule) and quantifies the lie budget of a rule-aware on-off attacker;

- Extension II, quorum voting over signed, round-bound vote vectors of replicated twins, under which, when the replicas share their evidence and honest votes arrive in time, up to $k-1$ corrupted replicas change no verdict, and missing or withheld votes never cause a new isolation;
- Extension III, a signed Merkle audit log of verdicts, votes and trust state that replicas endorse every K rounds off the decision path and that can be anchored in a permissioned ledger, with a tamper-evidence theorem that makes explicit when an external checkpoint is needed; and
- a reproducible evaluation, with parameters selected on a separate tuning seed, covering two calibrations to the original evaluation, long horizons, stealthy, minimal-footprint and rule-aware attackers, corrupted, withholding and slow replicas, bursty, common-mode, cross-factor and uneven congestion, a 100-sensor topology, and a trace-driven evaluation on two public packet-delivery datasets (Intel Lab and FlockLab). The simulator, configurations, raw results and figure scripts are publicly available (see Code availability).

The rest of the paper is organised as follows. Section 2 reviews related work, Section 3 formalises the problem, the threat model and the trust assumptions, Section 4 presents the three extensions with their analysis, Section 5 reports the experiments, Section 6 discusses them and compares the design with related frameworks, and Section 7 concludes.

2 Related Work

Digital twins. Tao *et al.* [2] review the state of the art of digital twins in industry, where a twin integrates the physical and cyber spaces of a product or process. Minerva *et al.* [3] widen the view to the IoT, and Sleuters *et al.* [14] use twins for the semi-automated behaviour analysis of large distributed IoT systems. Gehrmann and Gunnarsson [15] propose a twin-based security architecture for industrial control, Alcaraz and Lopez [8] catalogue DT security threats in a layered taxonomy that informs our threat model, and Empl *et al.* [16] prototype SOAR4IoT, a security orchestration, automation and response application built on twins.

Digital twins and blockchain. Onwubiko *et al.* [17] manage digital-twin documents with Ethereum smart contracts and IPFS storage: stakeholders approve document versions by a two-thirds threshold on chain and file hashes are anchored for traceability; the prototype is evaluated by gas cost and computes no trust score for devices. Sasikumar *et al.* [18] connect twins of an industrial IoT network through a proof-of-authority blockchain whose master nodes validate the network, and report lower energy consumption and better data security in simulation. TMIoDT [13] combines a digital-twin network, a blockchain-based credit score and a deep-learning intrusion detector for drone services; it is discussed in Section 6.1.

IoT trust management. Bao and Chen [5] argue for trust that adapts over time instead of a verdict computed once, Nitti *et al.* [6] carry the idea to the social IoT, and EigenTrust [19] gives the canonical recipe for continuous reputation by power iteration. The beta reputation system of Jøsang and Ismail [20] updates trust from counts of positive and negative observations with a forgetting factor. Sun *et al.* [7]

analyse on-off and bad-mouthing attacks on trust evaluation, and Chen, Bao and Guo [21] evaluate trust management for the social IoT under self-promoting, bad-mouthing, ballot-stuffing and discriminatory attacks. The surveys [22–24] review the field. Hussain *et al.* [25] propose a context-aware trust and reputation model for fog-based IoT in which a device's context is its role or purpose in the network and the trust level in $[0, 1]$ is updated recursively from previous values, with a weight of 0.7 on the current trust level, and multi-source feedback.

Fog and edge computing. Bonomi *et al.* [26] introduce fog computing, Chiang and Zhang [27] survey its research opportunities for the IoT, and Shi *et al.* [28] push computation further to the edge.

Blockchain for IoT and fog. Dorri *et al.* [9] build a lightweight chain for the smart home, and Novo [10] uses one for scalable access management. Debe *et al.* [29] add an Ethereum-based reputation system for public fog nodes. BlockEdge [11] integrates blockchain pre-processing into an industrial edge pipeline, and BPT [12] establishes trust between vehicles before a fog-computing service through a bid and rating exchange recorded on a chain.

Tamper-evident logs and anchoring. Tamper evidence does not require a blockchain. Haber and Stornetta [30] link time-stamped documents by hashes, Schneier and Kelsey [31] make audit logs on untrusted machines tamper-evident with hash chains, and Crosby and Wallach [32] use history trees to make such logs efficient to audit. Certificate Transparency [33] publishes signed Merkle tree heads whose leaf and interior hashes are domain-separated. The Keyless Signature Infrastructure (KSI) of Buldas *et al.* [34] publishes only the root of a hash tree, and Arbitrum [35] executes smart contracts off-chain while the chain records only hashes of their state. Extension III follows this line: it is a signed Merkle log that a ledger may host but whose guarantee does not depend on one. Section 6.1 compares the closest trust systems on the figures they actually report.

Network-level anomaly detection. N-BaIoT [36] detects botnet traffic with deep autoencoders. It observes traffic rather than behavioural trust factors and is complementary to the present work.

3 Problem Statement

For every pair of communicating nodes, the IoDT model of [4] evaluates four binary factors. Energy E is one when the energy a node consumed lies within the bounds $[E_{\min}, E_{\max}]$; $E_{\max}$ follows from the capacity of the node's super-capacitor and $E_{\min}$ from the CSMA/CA transmission and back-off cost, both scaled by the number of packets sent. The PDR score is one when the measured PDR reaches a threshold Θ_{PDR} (90% is the example value of [4]). Traceability T is one when the node's most recent claimed transaction is present in the trust database, and the twin-notification score W is one unless the twin of the destination named in the node's notification denounces the claimed communication (our reading of the notification check of [4]). The verdict is the logical AND

$$\mathrm{Trust}(i) = E(i) \wedge \mathrm{PDR}(i) \wedge T(i) \wedge W(i). \tag{1}$$

3.1 System Notation

Let $\mathcal{N}$ be the set of physical nodes and $\mathcal{N}_c \subset \mathcal{N}$ the unknown set of compromised nodes. Time is divided into evaluation rounds $t = 1, 2, \ldots$; in every round the trust engine computes, for every node i, the factor vector

$$\mathbf{x}_t(i) = \big(E_t, P_t, T_t, W_t\big)(i) \in \{0,1\} \times [0,100] \times \{0,1\}^2 \tag{2}$$

where P_t is the PDR in percent. A trust-aggregation function f maps the history $\mathbf{x}_{1:t}(i)$ to a verdict $\hat{y}_t(i) \in \{0,1\}$, with $\hat{y} = 0$ meaning "flagged".

3.2 Performance Metrics

The following metrics are used throughout.

- The *per-round false-positive rate* over a horizon of T rounds,

$$\mathrm{FPR} = \frac{1}{T\,|\mathcal{N} \setminus \mathcal{N}_c|} \sum_{i \notin \mathcal{N}_c} \sum_{t=1}^{T} \mathbb{1}[\hat{y}_t(i) = 0]. \tag{3}$$

- The *false-isolation rate*: the share of honest nodes flagged at least once within the horizon T. IoDT isolates a node as soon as it is flagged [4], so this is the number that matters for a run. It grows with T; the main tables use $T = 16$ rounds, the length of one reference scenario (Section 5.1), and Section 5.3 reports it for up to 1,024 rounds.
- The *mean time to first false isolation* (MTTFI) of an honest node over long operation, estimated as the inverse of the rate at which flags start on honest nodes; it does not depend on T. Strictly, this is the mean time between flag onsets; when flags are frequent it slightly overstates the time to the first one (about 8 against 7 rounds for the AND at $q = 0.04$), which does not change any comparison below.
- Under *quarantine with re-admission*, a flagged node is quarantined and re-admitted after R consecutive rounds in which a quorum of its twins reports $\tau_t \geq \theta_r$ and no persistence trigger ($R = 8$, $\theta_r = 0.9$); for the baselines, which have no τ_t, the condition is R consecutive rounds without a flag. We report the share of time an honest node spends in quarantine, the number of false quarantines per node and hour, and the share of rounds in which an attacker delivers a lie while it is not quarantined.
- The *detection rate*: the share of runs in which a compromised node is flagged at least once within T rounds, together with the mean *time to detection* (TTD) in rounds, and the *steady-state per-round detection*, the share of rounds $t \geq 3$ in which a compromised node is flagged.

3.3 Threat Model

Following the IoT threat survey of [1] and the DT threat taxonomy of [8], the adversary can compromise a subset $\mathcal{N}_c$ of physical nodes, corrupt at most c of the n twin replicas

Table 1 Trust assumptions and threat coverage.

Component / attack	Assumption or coverage
Fog relay	Trusted to relay evidence and to enforce isolation; a compromised fog node is out of scope
Twin replicas	Up to $k-1$ of n Byzantine: shield, frame or withhold (Sec. 5.5)
Trust engine	Trusted for timeliness; the enforced verdicts are checked against the signed votes at every header commit, so an unsupported verdict is exposed within K rounds plus the append time, but not prevented (Sec. 4.2)
Log storage, ledger	May delete or rewrite; rewrites are rejected by the verifier unless made with k replica keys before a newer checkpoint (Theorem 1); deleting the newest windows is detected only against a newer checkpoint
Checkpoint channel	Controlled independently of the cloud operator
Lies, stealthy lies	Covered for frequent lies; liars at $p \leq 0.25$ are detected slowly (Sec. 5.4)
Rule-aware on-off	Partly: a lie budget remains (Sec. 5.4)
Bad-mouthing via W	Not covered: a compromised destination can denounce an honest neighbour; repeated denunciations trigger the persistence check
Ballot-stuffing via W	Partly: a colluding destination can hide the W failure; under our lie model the lie still fails T, but a liar that also gets its transaction logged is not detected
Sybil, whitewashing	Out of scope: identities are provisioned by the operator [37, 38]
Denial of service	Detected in [4] mainly through abnormal energy use (a DoS against a twin); not re-evaluated here

that watch a node, steal replica signing keys, and rewrite or delete records in the cloud. The adversary acts in one of four ways.

- *Lie (IoDT scenario).* As in [4], the attacker reports a false destination to its twin, and the twin of the falsely named destination denounces the claim. We model a lie as setting $W = 0$ and, because the claimed transaction is not found in the database, also $T = 0$; the second effect is our modelling choice. The attacker lies in each round with probability $p \in (0, 1]$; $p = 1$ is the original naive attacker and $p < 1$ a stealthy one.
- *Minimal-footprint lie.* The attacker arranges for the claimed transaction to be logged, so a lie costs only $W = 0$, the smallest observable deviation.
- *Rule-aware on-off lie.* The attacker knows the aggregation rule and its parameters, observes its own past evidence, and lies only in rounds in which, absent congestion, no trigger would fire; once flagged it behaves honestly until it is re-admitted. This is the on-off attack of [7] against a known detector.
- *Corrupted twins.* The adversary controls c of the n replicas of each twin, for compromised and honest nodes alike. Corrupted replicas vote TRUST for the attackers (*shielding*), may also vote FLAG for honest nodes (*framing*), or may withhold their votes.

Attackers that never deviate on any factor are indistinguishable from honest nodes at this layer and are out of scope.

Table 1 lists which component the design trusts for what, and which attacks from the IoT trust literature it covers.

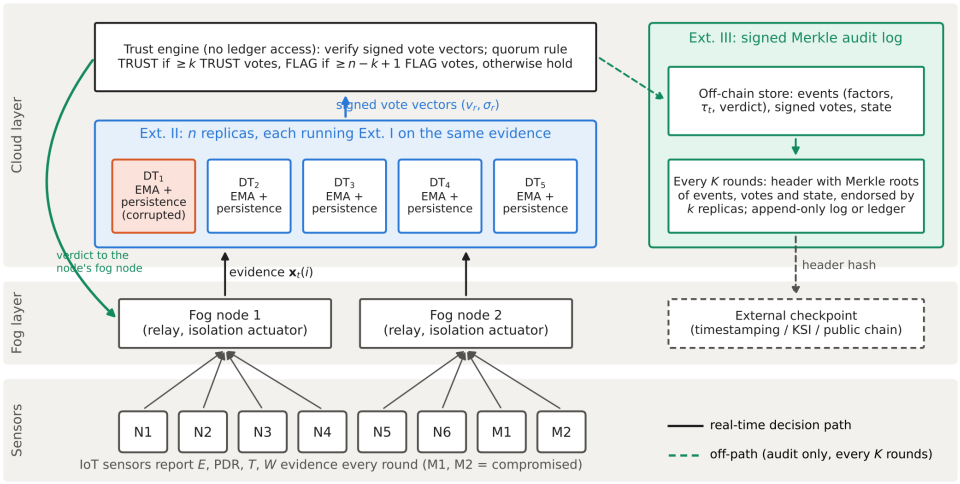


Fig. 1 Architecture of the proposed IoDT trust pipeline with its three extensions.

4 The Proposed Trust Model

This section presents the three extensions to the IoDT model [4], whose architecture is shown in Fig. 1. Twin replicas run Extension I, a quorum over their signed votes (Extension II) decides on the real-time path, and verdicts and votes are committed to a signed Merkle audit log off that path (Extension III).

4.1 Extension I: Continuous Trust Score

The four factors are mapped to $[0, 1]$ and combined with non-negative weights that sum to one:

$$s_t(i) = w_E E_t(i) + w_P \frac{P_t(i)}{100} + w_T T_t(i) + w_W W_t(i). \tag{4}$$

All experiments use $w_E = w_P = 0.30$ and $w_T = w_W = 0.20$, which put slightly more weight on the physical factors than on the notification-driven ones. A single bad round should not condemn a node that has otherwise behaved well, so the score is smoothed by an exponential moving average,

$$\tau_t(i) = \alpha\, s_t(i) + (1 - \alpha)\, \tau_{t-1}(i), \qquad \tau_0(i) = 1, \tag{5}$$

and the node is flagged when $\tau_t(i) < \theta$. The defaults are $\alpha = 0.5$ and $\theta = 0.70$, selected on a separate tuning seed (Section 5.1).

Smoothing has a blind spot: a weighted score cannot react to an attacker that always fails the *same* low-weight factor. An attacker that fails only W keeps $s_t \in [0.776, 0.80]$, and, absent congestion, its EMA never falls below 0.70. Extension I therefore adds a *per-factor persistence check*: the node is also flagged when any single factor has failed in at least m of the last w rounds. Congestion scatters failures across factors and rounds, whereas a liar fails the same factor again and again. The defaults are

Algorithm 1 Extension I trust update for node i at round t.

Input: factor scores (E_t, P_t, T_t, W_t); weights; prior trust $\tau_{t-1}(i)$; failure counters of the last w (and w_2) rounds; α, θ, m, w (and m_2, w_2).

Procedure:

1. $s_t \leftarrow w_E E_t + w_P(P_t/100) + w_T T_t + w_W W_t$.
2. $\tau_t(i) \leftarrow \alpha\, s_t + (1-\alpha)\, \tau_{t-1}(i)$.
3. Update, for each factor, the number of failures in rounds $t-w+1, \ldots, t$ (a factor fails when it is 0, or when $P_t < \Theta_{\mathrm{PDR}}$).
4. If $\tau_t(i) < \theta$, or some factor failed in $\geq m$ of the last w rounds (or in $\geq m_2$ of the last w_2), return FLAG; else return TRUST.

$m = 4$ and $w = 8$. A second, longer window (m_2 of w_2 rounds) can be added against rule-aware attackers (Section 5.4); it is off by default. Algorithm 1 gives the complete per-round update.

4.1.1 False-Positive Analysis

The analysis rests on three assumptions, and the experiments test what happens when the first two are relaxed.

(A1) *Independence.* For an honest node, factor corruptions are independent across factors and rounds.

(A2) *Symmetric corruption.* Each factor is corrupted with probability q; a corrupted binary factor becomes 0 and a corrupted PDR falls below Θ_{PDR}.

(A3) *Healthy-link PDR.* An uncorrupted PDR lies in $[\underline{P}, 100]$; the simulator uses $\underline{P} = 92\%$.

Under (A1)–(A2) the binary AND (1) flags an honest node whenever any factor is corrupted, so its per-round FPR is

$$\mathrm{FPR}_{\mathrm{AND}}(q) = 1 - (1-q)^4 = 4q + O(q^2), \tag{6}$$

and by the same counting any rule that flags only after two failures — the AND required to fail in two consecutive rounds, or two of the four factors required to fail in the same round — errs at rate $\Theta(q^2)$. In the evaluation the 2-of-4 rule is combined with the persistence check above, which keeps its order at $\Theta(q^2)$ and lets it see a liar that fails only one factor. These rules serve as delay-matched baselines in Section 5. The EMA needs three failures.

Proposition 1 (FPR of the EMA score) *A node is flagged by the EMA at round t if and only if*

$$s_t < s^\star = \frac{\theta - (1-\alpha)\, \tau_{t-1}}{\alpha}. \tag{7}$$

With the default weights, $\alpha = 0.5$ and $\theta = 0.70$, a flag requires at least three factor failures within the last $L = 6$ rounds, so under (A1)–(A3)

$$\mathrm{FPR}_{\mathrm{EMA}}(q) \le \binom{4L}{3} q^3, \qquad \mathrm{FPR}_{\mathrm{EMA}}(q) = \Theta(q^3). \tag{8}$$

The cubic order depends on the parameters and on (A1). At $\alpha \ge 0.6$ (with $\theta = 0.70$) or $\theta \ge 0.75$ (with $\alpha = 0.5$), two failures in one round can already cross the threshold and the order falls to $\Theta(q^2)$. When one congestion event fails several factors of a node at once, (A1) fails across factors, and the order is again $\Theta(q^2)$ (Section 5.7). The price of smoothing is one round of delay: without congestion, a node that lies in every round scores $s_t < 0.60$, so from $\tau_0 = 1$ its EMA is below 0.70 after two rounds, whereas the AND flags it in the first.

The persistence check adds false positives at a controlled rate and catches lies that the EMA cannot see.

Proposition 2 (Persistence check) *Under (A1)–(A2) the probability that the persistence check flags an honest node in a given round is at most $4\binom{w}{m}q^m$ (7.2×10^{-4} at $m = 4$, $w = 8$, $q = 0.04$). A node whose lies fail a given factor independently in each round with probability p is flagged within any window of w rounds with probability at least $\Pr[\mathrm{Bin}(w, p) \ge m]$; for $p = 1$ it is flagged by round m. A node that keeps at most $m - 1$ lies in every window of w rounds is never flagged by the check: its* lie budget *is $(m - 1)/w$, or $\min\{(m - 1)/w, (m_2 - 1)/w_2\}$ with a second window.*

4.2 Extension II: Multi-Twin Voting

A single twin is a single point of trust: if it is corrupted, the node it watches is never flagged. Extension II deploys n replicas of each twin and decides by a quorum of their signed votes; the reported configuration is $n = 5$, $k = 3$. This is one-shot voting over signed verdicts in the sense of Byzantine quorum systems [39], not a full Byzantine-fault-tolerant agreement protocol such as PBFT [40]: there is no multi-phase commit, view change or total order on operations. The extension relies on the following assumptions.

- (B1) *Independent hosting.* The replicas run on administratively independent hosts (different fog sites or cloud availability zones), so compromising one host does not compromise the others.
- (B2) *Authenticated, bound votes.* In every round each replica signs one vector of verdicts for all the nodes it watches, bound to its identity, the round number and the current key epoch (ECDSA over secp256k1 in our implementation). Keys are rotated by starting a new epoch, which invalidates signatures from retired keys.
- (B3) *Bounded delay.* Votes from honest replicas arrive within a known bound Δ; a vote that arrives later is treated as missing.
- (B4) *Identical evidence.* The fog node forwards the same evidence to every replica, so honest replicas reach identical verdicts.

Algorithm 2 Per-round vote for node i.

Input: evidence $\mathbf{x}_t(i)$; replicas $\mathrm{DT}_1, \ldots, \mathrm{DT}_n$; quorum k; previous verdict.
Procedure:

1. Each DT_r runs Algorithm 1 on $\mathbf{x}_t(i)$ and signs its vote vector (σ_r), bound to (r, t, epoch).
2. The trust engine waits at most Δ, verifies each received signature and its binding, and counts the TRUST votes V_T and the FLAG votes V_F.
3. If $V_T \geq k$ return TRUST; if $V_F \geq n - k + 1$ return FLAG to the fog node, which isolates node i; otherwise keep the previous verdict and raise an alarm.
4. Append the verdict and the signed votes to the off-chain store (input to Algorithm 3).

(B5) *Adversary budget.* At most $c \leq k - 1$ replicas are corrupted.

Each replica runs Algorithm 1, signs its vote vector and sends it to the trust engine, which verifies the signatures and applies the *quorum rule* of Algorithm 2: TRUST if at least k valid votes say TRUST, FLAG if at least $n - k + 1$ say FLAG, and otherwise the previous verdict is kept and an availability alarm raised. The engine forwards the verdict to the fog node, which enforces it and, being trusted (Table 1), appends the verdict it enforced to the off-chain store of Extension III. With all $n = 2k - 1$ votes present exactly one quorum is reached, so the rule coincides with "flag if fewer than k TRUST votes"; unlike that rule, missing or withheld votes never cause a new isolation, although a node that is already isolated stays isolated while no quorum forms. The count is computed by the trust engine, not by a smart contract, so no ledger operation lies on the decision path. The engine appends the signed votes to the same off-chain store.

Under (B3)–(B5) the quorum behaves like one honest twin. Corrupted replicas that always vote TRUST for an attacker (*shielding*) are outvoted whenever the $n - c \geq n - k + 1$ honest replicas flag it, and corrupted replicas that vote FLAG for an honest node (*framing*) are outvoted whenever the honest replicas trust it. Detection and false positives are therefore those of an honest single twin for every $c \leq k - 1$, whereas a single corrupted twin never flags its node and $c \geq k$ corrupted replicas decide every verdict.

The trust engine itself is not replicated. It is trusted to count promptly, but not trusted for integrity: every enforced verdict is logged with the signed votes behind it, and replicas endorse a header of the audit log only if each verdict in it follows from its votes by the quorum rule (Section 4.3). An engine that issues an unsupported verdict is therefore exposed when the next header is committed, within K rounds plus the time to append it, although it is not prevented in real time. Voting defends against corrupted twins; it does not help against attackers that lie rarely, whose detection is governed by Eq. (7) and Proposition 2. If the replicas receive independent evidence instead of (B4), framing replicas can tip honest nodes whose honest replicas disagree (Section 5.5).

4.3 Extension III: Signed Merkle Audit Log

Most blockchain-based trust schemes consult or update the chain while a decision is being made (Section 6.1). Here the verdict is produced by Extensions I and II alone, and an audit log records it afterwards. The log is a hash-chained sequence of signed headers over Merkle-committed off-chain data, in the tradition of tamper-evident logs [31–33]. Its headers can be kept in any append-only store; hosting them on a permissioned ledger (for example Hyperledger Fabric with a crash-tolerant ordering service among fog peers) adds replicated storage and availability against an operator that deletes the log, but the tamper evidence of Theorem 1 does not depend on the ledger's consensus.

Content. Every round, the per-node event (factor scores, τ_t, verdict; $|E| = 29$ bytes) and the replicas' signed vote vectors are stored off-chain. Every K rounds the trust engine proposes a header that carries the height, the first and last round, the numbers of events and votes, the key epoch, the previous header hash, and the Merkle roots of the window's events, of its signed votes and of the per-node state records ($|R| =$ 13 bytes per node); $|H| = 152$ bytes. The trees follow RFC 6962 [33]: leaf and interior hashes are domain-separated, the last node is not duplicated, and the leaf count is committed in the header. Each replica *endorses* the header — signs it — only after checking the off-chain data against it, including that every verdict follows from its votes; a header needs k endorsements. A partial final window is committed as well. Each header therefore occupies $|H|+k\,|\sigma|$ bytes in the header store, with $|\sigma| \leq 73$ bytes per endorsement (identifier and DER signature), independent of the number of nodes. Raw evidence never leaves the off-chain store, which also limits what the log reveals about each node. A verdict becomes auditable within K rounds plus the time to append a header; this delay never affects when the verdict itself is issued. We further assume:

(C1) header hashes are periodically published on an independently controlled channel (linked time-stamping [30], a KSI hash tree [34], an Arbitrum-style state commitment [35], or a public-chain anchor);
(C2) the signature scheme is existentially unforgeable and the hash function is collision resistant.

Definition 1 (Verifier) Given the headers and the off-chain data, the verifier V checks the hash links, at least k valid endorsements from distinct replicas of the header's key epoch, record lengths and counts, every Merkle root, every vote signature and its binding to replica, round and epoch, that each verdict follows from its round's votes by the quorum rule, that each state record equals the node's latest event, and, if a checkpoint $(j, h_j^\star)$ is available, that header j hashes to $h_j^\star$.

Theorem 1 (Tamper evidence) *Under (C2), let the adversary modify, insert, delete or reorder any event, vote or state record of a committed window B_i and recompute any hashes it likes.*

Algorithm 3 Audit-log commit at round t.

Input: pending events $\mathcal{E}$ and votes $\mathcal{S}$; state records; period K; previous header hash H_{prev}.

Procedure:

1. Append the round's events and signed votes (off-chain).
2. If $t \bmod K \neq 0$ and the run is not ending, return.
3. The trust engine forms the header with the Merkle tree hashes $\text{MTH}(\mathcal{E})$, $\text{MTH}(\mathcal{S})$, MTH(state), counts, epoch and H_{prev}.
4. Each replica checks the window (lengths, counts, roots, vote signatures and bindings, verdicts against votes, state against events) and, if it passes, signs the header.
5. With k endorsements, append the header; periodically publish its hash externally (C1).

1. *If it holds fewer than k replica signing keys, V rejects except with negligible probability.*
2. *If it holds k or more keys, it can make V accept if and only if no checkpoint $(j, h_j^\star)$ with $j \geq i$ was published before the modification (up to a hash collision).*

Endorsements, not the checkpoint, are therefore the first line of defence; the checkpoint is what defeats an adversary that steals or colludes with k replicas.

4.4 End-to-End Round

Fig. 2 shows how the three extensions interact in one round. Steps 1–7 form the real-time decision path: the fog node forwards the evidence to the replicas, each replica runs Algorithm 1 and signs its vote vector, and the trust engine applies Algorithm 2 and returns the verdict to the fog node, which isolates a flagged node. Steps 8–11 run afterwards and never delay a verdict: events and votes are appended to the off-chain store, and every K rounds Algorithm 3 forms a header, collects k replica endorsements and appends it; header hashes are periodically published on the checkpoint channel (C1).

5 Results

5.1 Experimental Setup

The extensions are evaluated with a discrete-round Python simulator of the IoDT factor model. It follows the topology of the reference scenario of [4]: eight sensors, two of which (COMP1 and COMP2, labelled M1 and M2 in Fig. 1) are compromised, attached to two fog nodes, with twins and the trust engine in the cloud. The simulator models the trust logic only; it is not a re-implementation of the iFogSim evaluation of [4], whose resource and energy model it does not include [41]. Delays are counted in evaluation rounds. Where a time is given, it assumes a round length $T_{\text{round}} = 117$ ms, obtained by dividing the 1875-ms execution time reported for the

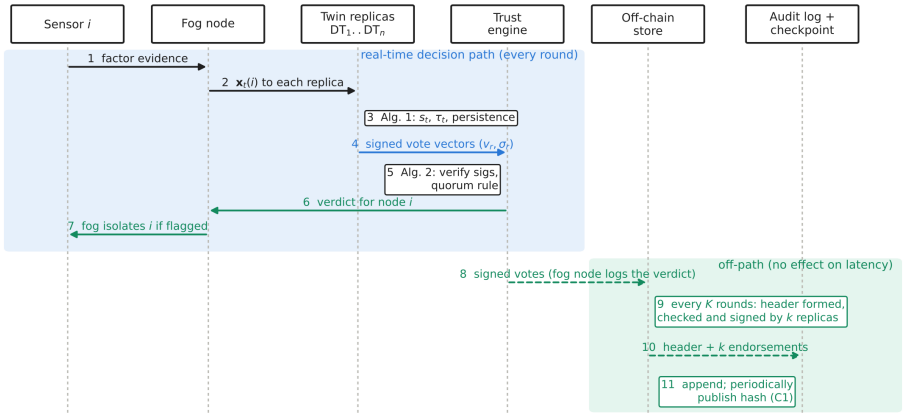


Fig. 2 Message sequence of one round: the decision path (steps 1–7) and the off-path audit-log commit (steps 8–11).

IoDT scenario in [4] into 16 rounds; the original model itself has no rounds, so this conversion is an assumption. A run lasts $T = 16$ rounds unless stated otherwise.

In every round each factor of every node is corrupted independently with probability q (assumptions (A1)–(A2)). Because the congestion level of a real deployment is unknown, results are reported over a range of q. Two levels are tied to the original evaluation, in which one of six honest nodes (N3) was misidentified in 2% of cases [4], a figure whose unit is not stated. *Reading A* takes it as a per-node, per-round rate of 2%/6, which the binary AND approximately reaches at $q = 0.001$ (per-round FPR 0.40% against 0.33%). *Reading B* takes it as a per-run false-isolation rate over the 16-round scenario, again divided among the six honest nodes (2%/6 per node), which the AND reaches at $q = 5.2 \times 10^{-5}$. Neither calibration reproduces the detection side: the simulated AND detects both attackers in every run, against 92% and 95% in [4], because a lie here always fails W and T, while the causes of the missed detections in the iFogSim runs are not reported. The stress level is $q = 0.04$. A healthy PDR is drawn uniformly from $[92, 100]$% and a degraded one from $[60, 89]$%, against the 90% threshold.

Extension I is compared with the binary AND of [4] and with two delay-matched rules: the AND required to fail in two consecutive rounds (*debounced AND*), which has the same one-round delay as Extension I, and a rule that flags when two of the four factors fail in the same round or the persistence check fires (*2-of-4 rule*). A beta reputation score [20] on the AND observation, tuned on seed 1 by an analogous rule (naive liar flagged within two rounds without congestion, lowest FPR at $q = 0.04$, near-ties resolved towards the simpler setting), reduced to the debounced AND (its FPR was within 0.002 percentage points in every cell of Table 3) and is not shown. The *full system* is Extension I in each of five replicas with a 3-of-5 quorum. Because its replicas receive identical evidence (B4), its verdicts coincided with those of Extension I in every cell of Table 3 (its row is therefore omitted), also with two corrupted replicas that shield the attackers and frame honest nodes; Section 5.5 evaluates corrupted replicas in detail.

Table 2 Default parameters of the evaluation.

Symbol	Meaning	Default
w_E, w_P, w_T, w_W	weights (4)	.3, .3, .2, .2
$\alpha,\ \theta$	EMA factor, threshold (5)	0.5, 0.70
$m,\ w$	persistence check	4 of 8 rounds
$m_2,\ w_2$	optional second window	off (8 of 32)
q	per-factor congestion rate	5.2×10^{-5}–0.04
T	horizon of a run	16 rounds
$R,\ \theta_r$	re-admission after	8 rounds with $\tau_t \geq 0.9$
$n,\ k$	replicas, quorum	5, 3
K	header (commit) period	10 rounds
Runs	Monte-Carlo runs per cell	2,000

The parameters of Extension I were selected on a tuning seed (seed 1) by two rules: the lowest per-round FPR at $q = 0.04$ among settings that flag a naive liar within two rounds on average, which selects $(\alpha, \theta) = (0.5, 0.70)$; and the lowest FPR among persistence windows that flag a persistent minimal-footprint liar within four rounds and detect one that lies in half of the rounds in at least 90% of runs, which selects 4 of 8. Every reported cell uses seed 0 with 2,000 runs unless stated otherwise (the long-horizon runs use seeds 100–105). Per-round rates carry 95% percentile-bootstrap intervals over runs, which respect the correlation between rounds of a run, and run-level proportions carry 95% Wilson intervals. For a zero count we give the one-sided 95% bound $3/N$ over the N honest node-runs (0.025% for $N = 12{,}000$), which also bounds the per-round FPR. Table 2 lists the defaults. The experiments ran in Python 3.13 with NumPy and the `cryptography` package (OpenSSL) on an Intel Xeon at 2.8 GHz, single-threaded. The tuning grids are included in the same repository.

5.2 Congestion Tolerance

Table 3 compares the aggregators at five congestion levels. Every aggregator detected both naive attackers in every run at every level, so the table reports what differs: false positives and delay.

Three regimes appear. At reading B the AND falsely isolates 0.39% of honest nodes within 16 rounds, close to the 0.33% that the reading targets, and every rule that needs more than one failure isolates none; there is little to improve. At reading A the AND isolates 6.2%, while the debounced AND and the 2-of-4 rule isolate 0.017% and 0.0083% (two and one of 12,000 honest node-runs) and Extension I none. At this level any rule that needs two failures removes almost all false isolations, and the 2-of-4 rule does so without delaying detection of a naive liar (a minimal-footprint liar still takes it about four rounds). Only as congestion grows does smoothing make the difference: at $q = 0.005$ Extension I still records no false flag where the debounced AND isolates 0.56% and the 2-of-4 rule 0.32%; at $q = 0.02$ the figures are 0.18%, 8.7% and 3.8%; and at $q = 0.04$ they are 1.1%, 26% and 14%, against 93% for the AND.

Fig. 3 shows why. The fitted log–log slopes of the per-round FPR are 0.99 for the AND (q from 0.0005 to 0.02), 2.0 for both two-failure rules (q from 0.002 to 0.04), and 3.1 for Extension I (q from 0.02 to 0.08; below 0.02 it records too few flags to fit),

Table 3 Congestion tolerance of Extension I and the baselines (2,000 runs, 16 rounds; rates to two significant figures, naive-liar TTD to two decimals; 95% intervals in brackets; zero counts with their one-sided 95% bound).

			TTD, naive	Minimal-footprint liar	
Aggregator	Per-round FPR %	False isolation %	liar (rounds)	Det. %	TTD (rounds)
$q = 5.2 \times 10^{-5}$ (reading B of [4])					
Binary AND [4]	0.024 [0.018, 0.032]	0.39 [0.29, 0.52]	1.00	100	1.0
Debounced AND	0 (<0.025)	0 (<0.025)	2.00	100	2.0
2-of-4 rule	0 (<0.025)	0 (<0.025)	1.00	100	4.0
Extension I	0 (<0.025)	0 (<0.025)	2.00	100	4.0
$q = 0.001$ (reading A of [4])					
Binary AND [4]	0.40 [0.37, 0.43]	6.2 [5.8, 6.6]	1.00	100	1.0
Debounced AND	0.0010 [0, 0.0026]	0.017 [0.0046, 0.061]	2.00	100	2.0
2-of-4 rule	0.00052 [0, 0.0016]	0.0083 [0.0015, 0.047]	1.00	100	4.0
Extension I	0 (<0.025)	0 (<0.025)	2.00	100	4.0
$q = 0.005$					
Binary AND [4]	2.0 [1.9, 2.0]	27 [26, 28]	1.00	100	1.0
Debounced AND	0.035 [0.027, 0.044]	0.56 [0.44, 0.71]	2.00	100	2.0
2-of-4 rule	0.020 [0.014, 0.026]	0.32 [0.23, 0.43]	1.00	100	3.9
Extension I	0 (<0.025)	0 (<0.025)	2.00	100	4.0
$q = 0.02$					
Binary AND [4]	7.8 [7.7, 7.9]	73 [72, 73]	1.00	100	1.0
Debounced AND	0.60 [0.56, 0.63]	8.7 [8.2, 9.2]	2.00	100	2.0
2-of-4 rule	0.25 [0.22, 0.27]	3.8 [3.5, 4.2]	1.00	100	3.7
Extension I	0.014 [0.0078, 0.020]	0.18 [0.11, 0.27]	1.98	100	4.0
$q = 0.04$ (stress level)					
Binary AND [4]	15 [15, 15]	93 [92, 93]	1.00	100	1.0
Debounced AND	2.1 [2.0, 2.2]	26 [25, 27]	2.00	100	2.0
2-of-4 rule	0.96 [0.92, 1.0]	14 [13, 15]	1.00	100	3.4
Extension I	0.097 [0.079, 0.12]	1.1 [0.92, 1.3]	1.96	100	3.9

in line with the $\Theta(q)$, $\Theta(q^2)$ and $\Theta(q^3)$ orders of Section 4.1. Equation (6) predicts 15.07% for the AND at $q = 0.04$, against 15.04% measured. The price of smoothing is one round: Extension I isolates a naive liar after 1.96–2.00 rounds, like the debounced AND, where the AND and the 2-of-4 rule need one. Extension I detects the minimal-footprint liar in every run at every level, after 3.9–4.0 rounds, as Proposition 2 predicts. Above $q \approx 0.08$ every aggregator degrades: at $q = 0.12$ Extension I falsely isolates 30% of honest nodes within 16 rounds.

5.3 Long Operation and Re-admission

A deployed trust engine runs for far longer than 16 rounds. Table 4 extends the horizon to 4,096 rounds (600 runs per cell, no attackers). False isolation grows with the horizon for every rule: at $q = 0.04$ Extension I falsely isolates 1.2% of honest nodes within 16 rounds in these runs (1.1% in Table 3), 20% within 256 and 60% within 1,024, while the debounced AND and the 2-of-4 rule reach 99% and 91% within 256 rounds.

The MTTFI does not depend on the horizon. At reading A it is 250 rounds for the AND (about 29 s at the assumed period), 6.2×10^4 rounds for the debounced AND (2.0 h) and 1.5×10^5 for the 2-of-4 rule (5.0 h). Extension I recorded no flag in 1.5×10^7 honest node-rounds, so its MTTFI is at least 4.9×10^6 rounds (6.7 days) at 95% confidence. At $q = 0.005$ its MTTFI is 7.4×10^5 rounds (24 h), against 2.6×10^3

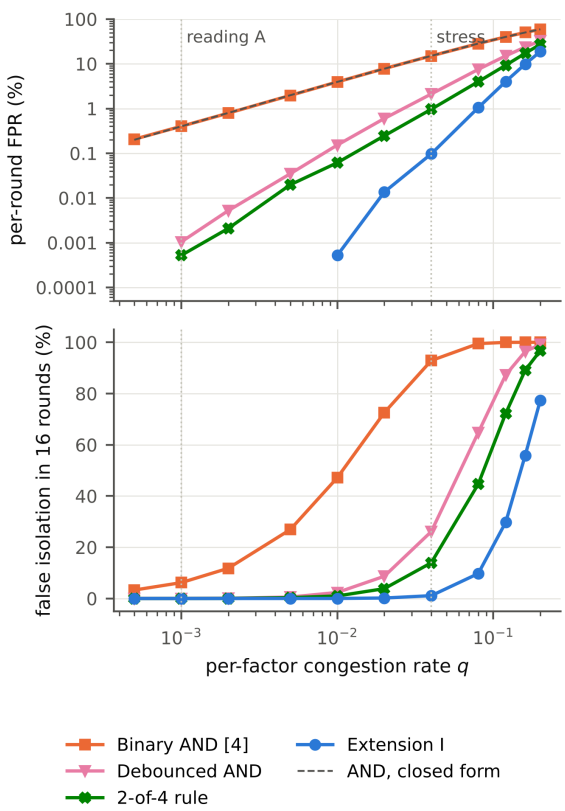


Fig. 3 Per-round FPR (top, logarithmic axes) and false isolation within 16 rounds (bottom) against the per-factor congestion rate, with dotted lines at reading A and at the stress level.

rounds (5 min) and 6.6×10^3 rounds (13 min) for the two simple rules; at $q = 0.04$ it is 1.1×10^3 rounds (2.2 min), against 52 and 110 rounds.

With quarantine and re-admission (Section 3.2), a falsely quarantined honest node returns after 8 to 18 rounds on average (9 to 16 under Extension I). At $q = 0.04$ an honest node then spends 1.4% of its time in quarantine under Extension I (27 false quarantines per node and hour), against 7.2%, 15% and 73% under the 2-of-4 rule, the debounced AND and the AND. At $q = 0.005$ the share is 0.0013% under Extension I and 0.12–15% under the others. Under sustained congestion as high as $q = 0.04$, false quarantines are an operating cost of every rule, which Extension I cuts by a factor of 5 to 50; at $q = 0.005$ it nearly removes them and at $q = 0.001$ it removes them.

5.4 Stealthy, Minimal and Rule-Aware Attackers

Table 5 varies the lying rate p of attackers that lie at random. Stealth has a price for every rule except the AND, which pays for its detection with false isolation. At reading A, Extension I detects a liar that fails T and W in 97% of runs at $p = 0.5$ but in 57% at $p = 0.25$ and 13% at $p = 0.10$ within 16 rounds, much like the debounced

Table 4 Long operation (600 runs of 4,096 rounds; no attackers): false isolation within 1,024 rounds, MTTFI, and quarantine with re-admission.

Aggregator	False isol. 1,024 rds %	MTTFI (rounds)	Quarant. time %	False quar. /node/h
$q = 0.001$				
AND [4]	98	250	3.2	119
Debounced AND	1.7	6.2×10^4	0.013	0.49
2-of-4 rule	0.58	1.5×10^5	0.0052	0.20
Extension I	0	$\geq 4.9 \times 10^6$	0	0
$q = 0.005$				
AND [4]	100	51	15	520
Debounced AND	33	2,600	0.31	12
2-of-4 rule	15	6,600	0.12	4.7
Extension I	0.14	7.4×10^5	0.0013	0.042
$q = 0.04$				
AND [4]	100	7.8	73	1260
Debounced AND	100	52	15	524
2-of-4 rule	100	110	7.2	265
Extension I	60	1,100	1.4	27

Table 5 Detection within 16 rounds (%) of attackers that lie at random with probability p (2,000 runs; the false isolation of each rule is that of Table 3).

	Lie fails T and W				Lie fails W only			
p	AND	Deb.	2-of-4	Ext. I	AND	Deb.	2-of-4	Ext. I
$q = 0.001$								
0.5	>99.9	96	>99.9	97	>99.9	96	92	92
0.25	99	55	99	57	99	55	33	32
0.1	83	14	82	13	83	14	2.8	2.2
$q = 0.04$								
0.5	100	99	>99.9	99	100	99	97	94
0.25	99.9	82	99	70	99.9	82	64	44
0.1	98	50	84	24	98	50	31	8.4

AND; over 64 rounds the last two figures rise to 97% and 46%. The 2-of-4 rule flags every lie that fails two factors and detects such a liar almost as well as the AND (82% of runs at $p = 0.10$), at the false-isolation cost of Table 3. A liar that fails only W is detected by Extension I in 92% of runs at $p = 0.5$ and 32% at $p = 0.25$ (86% over 64 rounds), about as often as by the 2-of-4 rule, whose persistence check is the same. At $q = 0.04$ every rule detects more, partly because congestion adds failures to the attacker's own record.

A rule-aware attacker does better (Table 6). The simulated attacker predicts its own EMA from its past evidence (assuming a 96% PDR), keeps a margin of 0.02 above θ, and respects every persistence window. Without congestion it lied in 3 of 8 rounds (37.5%) against Extension I and in 7 of 32 (21.9%) with a second window of 8 failures in 32 rounds, without being flagged — the budgets of Proposition 2. Every lie still

Table 6 Rule-aware on-off attacker tuned to Extension I (all defences face the same strategy): lie budget without congestion, detection within 256 rounds at reading A (500 runs), lies delivered per round with re-admission (400 runs of 2,048 rounds, T and W fail), and the honest nodes' quarantine time at $q = 0.04$.

	Budget %		Det. %		Delivered %		Hon.
Defence	TW	W	TW	W	q=.001	q=.04	quar. %
Extension I	37.5	37.5	37	20	36	19	1.4
Ext. I + 8 of 32	21.9	21.9	36	20	22	14	1.4
AND [4]	0	0	100	100	0.21	2.2	73

fails W, so the AND flags the first one, at a cost of 6.2% false isolation at reading A. At reading A congestion rarely adds failures: over 256 rounds Extension I detects the attacker in 37% of runs (T and W) and 20% (W only), and with re-admission the attacker delivers a lie in 36% of rounds (22% with the second window). At $q = 0.04$ congestion pushes the attacker over a trigger more often, and the delivered share falls to 19% (14%); a rule-aware liar that fails only W delivers 25% (16%). The simulated attacker is tuned to Extension I and faces every defence with the same strategy, so the AND and debounced-AND figures show what that attacker suffers, not what an attacker tuned to them would achieve; against the debounced AND it delivered 16% of rounds (T and W) and 4.5% (W only) at $q = 0.04$. Analytically, the simple rules leave budgets of their own: the debounced AND never sees lies that are one round apart (budget 50%), and the 2-of-4 rule flags every lie that fails T and W but leaves a W-only liar the same 37.5% budget. Threshold rules with published parameters thus leave a lie budget that a second, longer window shrinks but does not remove.

5.5 Corrupted, Withholding and Slow Replicas

Table 7 turns to Extension II at $q = 0.04$, with identical evidence for all replicas (B4). A corrupted single twin hides its node in every run. With 3-of-5 voting, up to $k-1 = 2$ corrupted replicas that shield the attackers and frame honest nodes change nothing: detection stays at 100% of runs and of steady-state rounds, and the FPR and false isolation equal those of an honest single twin, as Section 4.2 argues. A third corrupted replica ($c = k$) defeats the vote: the attackers are never flagged and every honest node is isolated. Deployments that expect three replicas at risk need 4-of-7, which tolerates them.

Missing votes are handled by the quorum rule. With two withholding replicas plus 5% vote timeouts, or with 20% timeouts, it keeps false isolation at the level of an honest single twin and detection at 100% of runs; the cost is rounds with no quorum (14% and 5.8%), in which verdicts are held and an alarm is raised. With 20% timeouts and two withholding replicas, half of all rounds have no quorum and per-round detection falls to 97%, but no missing vote caused a new isolation. These timeout rows relax (B3); a node that is already isolated stays isolated, and cannot be re-admitted, while no quorum forms.

Table 7 Corrupted, withholding and slow replicas at $q = 0.04$, identical evidence (2,000 runs; per-round detection in steady state).

Setting	Det. %	Per-rd det. %	FPR %	False isol. %	No quor. %
Single twin, honest	100	100	0.097	1.1	–
Single twin, corrupted	0	0	0.097	1.1	–
3-of-5, $c = 2$ shield + frame	100	100	0.097	1.1	0
3-of-5, $c = 3$ shield + frame	0	0	100	100	0
4-of-7, $c = 3$ shield + frame	100	100	0.097	1.1	0
$c = 2$ withhold, 5% timeouts	100	99.8	0.096	0.98	14
20% timeouts	100	>99.9	0.096	1.0	5.8
$c = 2$ withhold, 20% timeouts	100	97	0.091	0.67	49

Table 8 Audit-log storage for 100 sensors over 30 rounds.

K	Headers	Header bytes	Off-chain (B)	Max header (B)
1	30	11,045	152,028	371
5	6	2,205	120,834	370
10	3	1,107	116,932	369
20	2	734	115,620	367

(B4) matters. When each replica instead misreads each factor independently with probability η on top of the shared congestion ($q = 0.04$), five honest replicas lower false isolation from 12% (one twin) to 2.4% at $\eta = 0.05$, but replicas that frame honest nodes now tip the nodes whose honest replicas disagree: false isolation rises to 6.0% with one framing replica and 29% with two, and to 72% with two at $\eta = 0.10$ (against 9.3% with five honest replicas). Shielding costs little: with two shielding replicas, steady-state detection falls from 99.9% to 98% at $\eta = 0.05$. The "honest twin" guarantee of Extension II therefore holds only when the replicas see the same evidence.

5.6 Audit Log: Cost and Tamper Evidence

Table 8 reports the audit log for 100 sensors over 30 rounds of simulator output, with one corrupted replica and 5% vote timeouts. A header with three endorsements occupies 367 to 371 bytes (DER signatures vary slightly in length) whatever the number of nodes, so header storage falls from 11.0 kB at $K = 1$ to 1.1 kB at $K = 10$; at the assumed period and $K = 10$ this is about 27 MB a day for any fleet size. Off-chain, the events, signed vote vectors and state records of the 30 rounds occupy 115–152 kB, of which 87 kB are events.

Table 9 reports 30 random edits per cell at $K = 10$. The verifier rejected every edit when the adversary held fewer than k keys, whether it left the hashes untouched, recomputed them, or re-signed with $k - 1$ stolen keys. Substituting interior-node values for events and replaying a vote were rejected even with k keys, because record lengths and vote bindings are checked. The other k-key rows measure our implemented forger, not a limit of the scheme: by Theorem 1 a forger that re-signs every affected

Table 9 Edits rejected by the verifier (%, 30 per cell, $K = 10$; the "older" column uses the 20–21 trials whose edit follows an earlier header) by edit and adversary; "–": not run.

	No keys		$k-1$	k keys, checkpoint		
Edit	edit	rehash	keys	none	older	newer
Verdict (with its votes)	100	100	100	10	10	100
Trust value τ_t	100	100	100	0	0	100
State record	100	100	100	0	0	100
Signed vote	100	100	100	0	0	100
Interior-node substitution	100	–	–	100	100	100
Duplicated event	100	–	–	100	100	100
Truncated window	100	–	–	100	100	100
Replayed vote	100	–	–	100	100	100

record passes whenever no newer checkpoint exists. Our forger duplicated events and truncated windows without re-signing the affected counts and votes, so it was rejected at $K = 10$, whereas at $K = 1$ a truncation passed without a checkpoint. It rewrote a trust value, a state record or a vote and passed without a checkpoint or with an older one, and was always rejected against a newer one. Its verdict edits were rejected without a checkpoint in 10% of trials, all of them rounds in which a stolen replica's vote had timed out and the forger did not sign a replacement. Finally, in 30 trials a trust engine that wrote a verdict its votes did not support never obtained the k endorsements needed to commit the window. These trials check the implementation; the guarantee itself rests on the cryptographic assumptions.

5.7 Robustness

Fig. 4 relaxes assumptions (A1) and (A2) in four ways.

Bursts. Holding the mean congestion at $q = 0.04$, a two-state Markov chain alternates between $q = 0.02$ and $q = 0.20$ (one ninth of the time in the high state) with mean bursts of 1 to 32 rounds. Longer bursts raise Extension I's false isolation from 3.7% to about 9% when each node has its own chain (panel a), and from 4.2% to about 9.5% when one chain governs the whole network. The AND (77–91%), the debounced AND (18–29%) and the 2-of-4 rule (16–32%) stay far higher in either case.

Common-mode events. An event that knocks out all factors of every node behind a fog node, with probability q_{corr} per round, cannot be told from an attack on all those nodes at once, and Extension I's false isolation rises to 8.4% at $q_{\text{corr}} = 0.005$ and 28% at 0.02 (panel b). Because such events hit many nodes together, we evaluated *domain suppression*: when more than half of a fog domain is flagged in the same round, those flags are discarded for that round (an alarm would be raised; it is not modelled). It cuts false isolation to 1.6% and 3.5% at the same rates and left the detection of both attackers at 100% of runs. An adversary able to provoke common-mode events could use suppression to mask its own lies during them; that case is not evaluated.

Cross-factor events. When one node-level event fails P, T and W together (4,000 runs per point, no other noise), (A1) fails across factors and Extension I errs at $\Theta(q^2)$ (panel c): its FPR grows from 0.011% to 0.055% to 0.22% as the event rate doubles

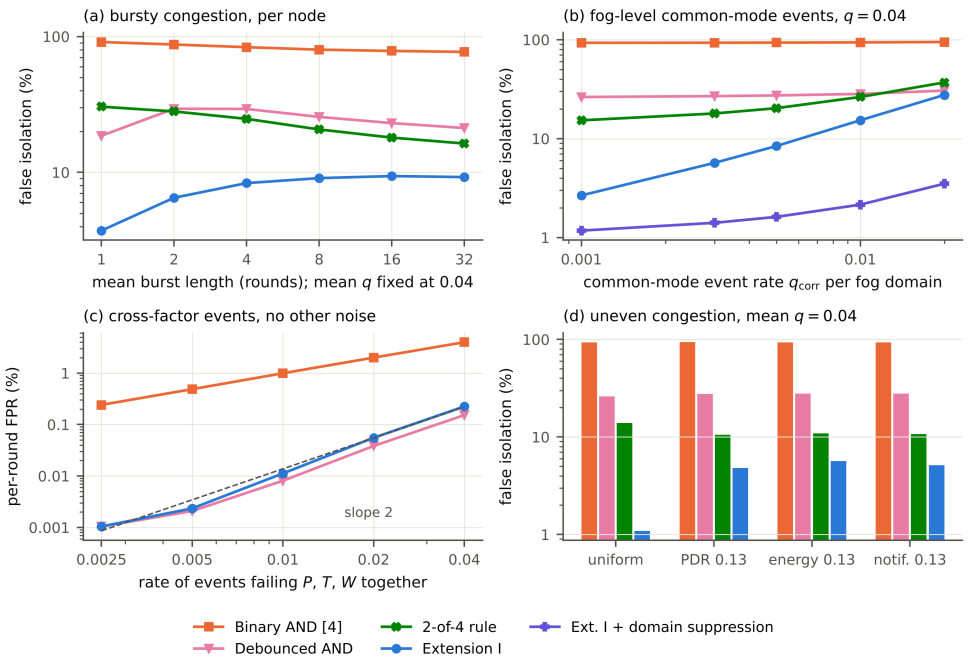


Fig. 4 Robustness to bursty (a), common-mode (b), cross-factor (c) and uneven (d) congestion.

from 0.01 to 0.04. The debounced AND is then better at every rate (0.15% against 0.22% at 0.04), and the 2-of-4 rule (not plotted) is as bad as the AND, since every event fails three factors. Such an event also looks exactly like a lie that fails T and W with a degraded PDR.

Uneven congestion. Concentrating the same mean congestion on one factor (0.13 on it, 0.01 on the others) leaves the AND almost unchanged (15.0% to 15.6% per round) but raises Extension I's per-round FPR eight- to ninefold, to 0.81–0.88%, and its false isolation to 4.8–5.7% (panel d), because one factor now fails repeatedly and the persistence check fires. Extension I stays below the 2-of-4 rule (11%), but a deployment that knows its noisiest factor should set m per factor.

Scale. With 100 sensors behind five fog nodes (500 runs, $q = 0.04$), Extension I isolated 1.1% of honest nodes, as in the eight-node case, and with common-mode events at $q_{\text{corr}} = 0.005$ domain suppression brought false isolation from 9.0% back to 2.0%.

5.8 Trace-Driven Evaluation on Public Datasets

The synthetic congestion of Sections 5.2–5.6 is independent across factors, rounds and nodes. To test the extensions on real loss patterns, we replaced it, for every node, with the packet-delivery record of a real deployment. The Intel Lab trace [42] records which readings of 54 Mica2Dot motes reached a base station, one per mote every 31 s, over 36 days of a lossy multi-hop network (51% of readings arrived; timestamp jitter

Table 10 False isolation within 16 rounds (%) on two public packet-delivery traces by retry count r, with q the share of lost factor messages in the evaluated windows (1,000 trace runs; 2,000 runs for the i.i.d. column).

							Ext. I	
Trace	r	q %	AND	Deb.	2-of-4	Ext. I	typ.	i.i.d.
Intel Lab	4	16	69	46	42	31	22	52
	8	7.9	42	24	22	15	6.6	9.6
	16	3.8	21	11	10	6.5	0.91	1.0
FlockLab	1	5.5	48	22	30	8.3	0.67	3.2
	2	3.2	35	8.4	6.8	2.7	0.074	0.60
	4	1.9	8.0	2.6	3.1	2.0	0.15	0.16

is not counted as loss, and periods in which almost no mote was heard are logging gaps and are skipped). The FlockLab dataset [43] records, about every two hours in August 2019, which of 100 back-to-back packets each of the 27 TelosB nodes of the FlockLab testbed [44] received from every other. We took as fog gateways the two nodes that put the most other nodes in range, and let each of the 24 remaining nodes in range report to the gateway it hears best (single hop, mean packet reception 95%). Two of these nodes ran only during the first day and a half, so FlockLab windows use 22 nodes and span the whole month.

Each factor's evidence of a node in a round is one message, sent in up to r attempts that use consecutive transmission opportunities of that node in the trace (31-s slots for Intel Lab, packets of one test for FlockLab); the factor fails when all r attempts are lost. The retry count r thus sets the loss level, which is then shaped entirely by the trace. The two traces differ in time scale: an Intel Lab round lasts $4r$ slots (8 to 33 minutes), whereas a FlockLab round spans a few milliseconds of back-to-back packets and windows that cross a test boundary join tests about two hours apart. A round in which no evidence of a node arrives at all is treated as unavailability, and every rule keeps its previous verdict (the *silence rule*). Each run draws a window of 16 rounds and 32 (Intel Lab) or 22 (FlockLab) nodes alive throughout, two of which are attackers whose lies are added to their own real losses (1,000 runs per cell, drawn from 73 to 309 distinct windows). No parameter was retuned. Windows start on a grid of 16 rounds, and the intervals below resample whole windows.

Table 10 gives the result. Extension I has the lowest false isolation in every cell, and the window-level interval of its difference to each other rule excludes zero in every cell (the smallest is 0.16 to 1.1 percentage points, against the debounced AND at FlockLab with $r = 4$). The traces narrow its lead, however. At Intel Lab with $r = 8$ (7.9% of messages lost) it falsely isolates 15% [12, 19] of honest nodes within 16 rounds, against 22–24% for the two-failure rules and 42% for the AND; with $r = 16$ the figures are 6.5% [4.2, 9.1], 10–11% and 21%. At FlockLab without retransmission it isolates 8.3% [7.5, 9.2] against 22–30% and 48%, and with $r = 4$ 2.0% [1.4, 2.7] against 2.6–3.1% and 8.0%. Independent congestion at the same mean rate (column "i.i.d.") would make the AND isolate far more nodes everywhere and Extension I fewer everywhere except

at the highest loss level (Intel Lab, $r = 4$). Two properties of real loss explain this. It is concentrated: at Intel Lab with $r = 8$ the median node loses 5.6% of its messages over the whole trace and the lossiest tenth 19%, so the AND's errors fall on a minority of nodes. And it is bursty across factors and rounds: in the evaluated windows two or more factors of a node fail in the same round 1.4 to 9.2 times as often as independent losses would make them, and a failed factor fails again in the next round in 42–76% of cases, which is the cross-factor case of Section 5.7, in which smoothing errs at $\Theta(q^2)$. For the nodes whose long-run loss is at most the median (column "typ.", about 10,000 to 15,500 node-runs per cell), Extension I's false isolation is 6.6% and 0.91% at Intel Lab ($r = 8$, 16) and below 0.7% at FlockLab; 65–99% of its false isolations fall on nodes with above-median loss.

Detection is slightly lower than in the synthetic study. Extension I flagged a naive liar in 98.6–99.9% of runs after about two rounds, and a minimal-footprint liar in 98.8–99.9% after 3.7–4.1 rounds. The misses are attacker nodes whose evidence was lost in almost every round of the window, so that their verdict stayed at TRUST under the silence rule; counting silent rounds as failures instead detects every attacker and raises false isolation by at most 2.4 percentage points, with Extension I still the lowest in every cell.

Over longer windows false isolation grows for every rule: within 128 rounds (Intel Lab, $r = 8$) and 256 rounds (FlockLab, $r = 1$) Extension I isolates 30% and 40% of honest nodes against 41–66% and 61–81% for the other rules, and with re-admission honest nodes spend 14% and 11% of their time in quarantine against 15–37%. The rule-aware attacker, still tuned to Extension I, delivered a lie in 28% (Intel Lab) and 27% (FlockLab) of rounds against it, 17% with the second window, and was detected in 61–74% of runs; the AND and the 2-of-4 rule held it to about 1.5% of rounds at the cost of the false isolation above. The traces thus confirm the ordering of the synthetic study but not its magnitude: on real, lossy links every rule isolates honest nodes far more often, and per-node baselines that separate a weak link from misbehaviour are needed before isolation is automated.

5.9 Latency

Table 11 gives the measured cryptographic cost per round. Because each replica signs one vote vector per round, the engine verifies five signatures, 2.0 ms for 8 or 100 nodes, and each replica signs once (0.5 ms); verifying one vote per node would cost the engine 203 ms per round at 100 nodes, more than the assumed 117-ms period. One EMA update for one node costs 0.1 μs; the persistence check was not timed separately. The remaining term is the time d_{rep} for votes to reach the engine, which depends on replica placement and was not measured: replicas in different availability zones of one cloud region are a single-digit number of milliseconds apart [45], whereas replicas at different fog sites add a fog-to-cloud link (100 ms in the configuration of [4]) unless the engine runs at the fog. Extension III adds nothing to the decision path. Relative to the AND, isolating a naive liar therefore takes one more round of evidence plus about 2.5 ms plus d_{rep}. All the added work runs in the fog and cloud, so the extensions add no computation or energy on the sensors.

Table 11 Measured per-round cost on the decision path (ms, $n = 5$ replicas, single thread).

Operation	8 nodes	100 nodes
EMA update, one node	0.0001	
Replica: sign one vote vector	0.52	0.48
Engine: verify 5 vote vectors	2.0	2.0

6 Discussion

6.1 Comparison With Related Frameworks

Table 12 positions the proposed system against the trust frameworks closest to it. Each cell reflects what the cited paper describes, and the latency row reports what each paper measures. Temporal smoothing is marked ✓ for an explicit forgetting factor and ∘ for trust that accumulates history without one. The latency figures are heterogeneous (trust setup, block operations, detect-and-isolate time) and come from different platforms, so they show where each system places its costs rather than ranking the systems. Within this set, the proposed design is the only one that combines a continuous, smoothed trust score, replicated twins with voting, and a tamper-evident record kept off the decision path.

TMIoDT [13] is the closest precedent. It assigns each drone a blockchain-recorded credit score and validates transactions by proof-of-authority consensus, so the chain is involved in its trust decisions, and it detects intrusions with deep models trained on labelled traffic (about 99.1% accuracy on ToN-IoT); its twins are not replicated. The proposed score needs no training data and keeps the log off the decision path, but sees only the four behavioural factors, so the two approaches are complementary.

6.2 Limitations

Behaviour-based trust has hard limits, which the experiments make explicit. An attacker that knows the rule keeps a lie budget (37.5% of rounds against Extension I without congestion, 21.9% with a second window), low-rate random liars are detected slowly, and one that never deviates on any factor cannot be seen at this layer at all. Bad-mouthing through the notification factor is not covered. Common-mode events need domain suppression, which an adversary able to provoke them could abuse, and events that fail several factors at once lower the order of the error to $\Theta(q^2)$. On the architecture side, the fog relay is trusted (it enforces verdicts and logs the verdicts it enforced), although a replica hosted at a fog site may be corrupted, and the trust engine is trusted for timeliness; its integrity is checked after the fact, within K rounds. Extension II assumes identical evidence and bounded vote delay; with independent replica errors, two framing replicas raise false isolation several-fold. An adversary that controls k replicas defeats any quorum by construction. Finally, the evaluation is a simulation of the trust logic; honest losses come from synthetic congestion or from two public delivery traces mapped onto the factors by our retransmission model, attackers are simulated, and on those traces lossy nodes are falsely isolated often under every

Table 12 Comparison with related trust frameworks (✓ yes, ○ partial or optional, – no).

Capability	BPT [12]	Debe [29]	Hussain [25]	Onwubiko [17]	TMIoDT [13]	IoDT [4]	Proposed
Digital twin	–	–	–	✓	✓	✓	✓
Continuous trust score	✓	✓	✓	–	○	–	✓
Temporal smoothing	○	○	✓	–	○	–	✓
Replicated twins, voting	–	–	–	–	–	–	✓
Blockchain	✓	✓	–	✓	✓	–	○
Chain off decision path	○	–	n/a	–	–	n/a	✓
Tamper-evident record	✓	✓	–	✓	✓	–	✓
Reported latency (what it measures)	2.47 s trust setup	n.r.	n.r.	n.r. (gas cost only)	15–16 ms block op.	250–300 ms detect + isolate	+1 round $+2.5\,\text{ms} + d_{\text{rep}}$

n.r. = not reported; n/a = not applicable (no blockchain).

rule; under the silence rule a node that goes completely silent keeps its last verdict, so silence itself must be monitored separately. The round length, the header append latency and the replica delay d_{rep} are assumptions or were not measured, and no ledger was deployed.

7 Conclusion

The IoDT model of [4] pairs every IoT node with a cloud-resident twin and decides trust from four interpretable factors. Its binary AND, single twin and mutable verdict store make it fragile under congestion, blind to a corrupted twin, and not tamper-evident. This paper kept the factor set and added three extensions: a weighted, EMA-smoothed score with a per-factor persistence check; quorum voting over signed vote vectors of replicated twins; and a signed Merkle audit log of verdicts and votes, endorsed by the replicas off the decision path.

The evaluation compared the score with delay-matched baselines rather than with the AND alone, and the comparison changes the claim. At the congestion levels implied by the original evaluation, any rule that needs two failures removes almost all false isolations, and a 2-of-4 rule does so without delaying the detection of a naive liar. Smoothing pays off as congestion grows and over long operation: at 4% per-factor congestion Extension I falsely isolated 1.1% of honest nodes within 16 rounds, against 14–26% for the simple rules and 93% for the AND, its mean time to the first false isolation was 110 times longer than that of the better simple rule at 0.5% congestion and 10 times longer at 4%, and with re-admission honest nodes spent 1.4% of their time in quarantine at 4%, at the cost of one extra round of evidence. An operator who knows that congestion is low can therefore use the simpler 2-of-4 rule; one who does not, or who expects congestion to vary, gains from smoothing. On two public packet-delivery traces, Extension I kept the lowest false isolation in every setting, but real losses, bursty and concentrated on a minority of nodes, raised it to 2.0–31% within 16 rounds, mostly on nodes with above-median loss. When the replicas of a twin see identical evidence, a quorum rule kept detection at the level of an honest twin with up to two of five replicas corrupted, and missing or withheld votes caused no new isolation; with independent replica errors, framing replicas raise false isolation markedly. The audit log rejected every edit by an adversary with fewer than k replica keys, rejected interior-node substitutions and replayed votes even with k keys, and exposed unsupported verdicts; its header size is independent of the number of nodes.

The results are bounded by the limitations listed in Section 6.2: a rule-aware attacker keeps a lie budget of 37.5% of rounds without congestion (21.9% with a second window), bad-mouthing through the notification factor is not covered, common-mode events need domain suppression, cross-factor events lower the order of the error from $\Theta(q^3)$ to $\Theta(q^2)$, voting relies on identical evidence, the fog relay is trusted, lossy nodes are falsely isolated often on real traces, and the evaluation simulates the trust logic with assumed timing and no deployed ledger. Future work will (i) replace fixed thresholds with sequential tests such as CUSUM [46] whose parameters are randomised, to shrink the lie budget; (ii) corroborate notification failures across several destinations against bad-mouthing; (iii) calibrate thresholds to each node's own loss history so

that a weak link is not taken for misbehaviour; (iv) stack the behavioural score with a traffic-level detector such as [36]; (v) deploy the replicas and the log on a multi-domain fog testbed to measure d_{rep} and the append latency; and (vi) couple the audit log with a SOAR platform [16] for automated response.

Data availability

The trace-driven experiments use the Intel Lab data [42] and the FlockLab link-quality dataset [43]. The raw results of every experiment, with the configuration of every run, and the pre-processed traces are available at https://github.com/Abdulelah7/IoDT.

Code availability

The simulator, the experiment, table and figure scripts and the trace pre-processing scripts are available in the same repository (https://github.com/Abdulelah7/IoDT).

References

[1] Neshenko, N., Bou-Harb, E., Crichigno, J., Kaddoum, G. & Ghani, N. Demystifying IoT security: an exhaustive survey on IoT vulnerabilities and a first empirical look on Internet-scale IoT exploitations. *IEEE Commun. Surv. Tutor.* **21**, 2702–2733 (2019).

[2] Tao, F., Zhang, H., Liu, A. & Nee, A. Y. C. Digital twin in industry: state-of-the-art. *IEEE Trans. Ind. Inform.* **15**, 2405–2415 (2019).

[3] Minerva, R., Lee, G. M. & Crespi, N. Digital twin in the IoT context: a survey on technical features, scenarios, and architectural models. *Proc. IEEE* **108**, 1785–1824 (2020).

[4] Soula, M., Mbarek, B. & Meddeb, A. A real-time trust management model using digital twin in IoT networks. *IEEE Access* **12**, 183326–183343 (2024).

[5] Bao, F. & Chen, I.-R. Dynamic trust management for Internet of Things applications. in *Proc. 2012 Int. Workshop on Self-Aware Internet of Things* 1–6 (ACM, 2012).

[6] Nitti, M., Girau, R. & Atzori, L. Trustworthiness management in the Social Internet of Things. *IEEE Trans. Knowl. Data Eng.* **26**, 1253–1266 (2014).

[7] Sun, Y. L., Han, Z., Yu, W. & Liu, K. J. R. A trust evaluation framework in distributed networks: vulnerability analysis and defense against attacks. in *Proc. IEEE INFOCOM 2006* 1–13 (IEEE, 2006).

[8] Alcaraz, C. & Lopez, J. Digital twin: a comprehensive survey of security threats. *IEEE Commun. Surv. Tutor.* **24**, 1475–1503 (2022).

[9] Dorri, A., Kanhere, S. S., Jurdak, R. & Gauravaram, P. Blockchain for IoT security and privacy: the case study of a smart home. in *Proc. IEEE Int. Conf. Pervasive Computing and Communications Workshops (PerCom Workshops)* 618–623 (IEEE, 2017).

[10] Novo, O. Blockchain meets IoT: an architecture for scalable access management in IoT. *IEEE Internet Things J.* **5**, 1184–1195 (2018).

[11] Kumar, T. *et al.* BlockEdge: blockchain-edge framework for industrial IoT networks. *IEEE Access* **8**, 154166–154185 (2020).

[12] Dewanta, F. & Mambo, M. BPT scheme: establishing trusted vehicular fog computing service for rural area based on blockchain approach. *IEEE Trans. Veh. Technol.* **70**, 1752–1769 (2021).

[13] Akram, J., Anaissi, A., Rathore, R. S., Jhaveri, R. H. & Akram, A. Digital twin-driven trust management in Open RAN-based spatial crowdsourcing drone services. *IEEE Trans. Green Commun. Netw.* **8**, 1061–1075 (2024).

[14] Sleuters, J., Li, Y., Verriet, J., Velikova, M. & Doornbos, R. A digital twin method for automated behavior analysis of large-scale distributed IoT systems. in *Proc. 14th Annual Conf. System of Systems Engineering (SoSE)* 7–12 (IEEE, 2019).

[15] Gehrmann, C. & Gunnarsson, M. A digital twin based industrial automation and control system security architecture. *IEEE Trans. Ind. Inform.* **16**, 669–680 (2020).

[16] Empl, P., Schlette, D., Zupfer, D. & Pernul, G. SOAR4IoT: securing IoT assets with digital twins. in *Proc. 17th Int. Conf. Availability, Reliability and Security (ARES)* 1–10 (ACM, 2022).

[17] Onwubiko, A., Singh, R., Awan, S., Pervez, Z. & Ramzan, N. Enabling trust and security in digital twin management: a blockchain-based approach with Ethereum and IPFS. *Sensors* **23**, 6641 (2023).

[18] Sasikumar, A. *et al.* Blockchain-based trust mechanism for digital twin empowered Industrial Internet of Things. *Future Gener. Comput. Syst.* **141**, 16–27 (2023).

[19] Kamvar, S. D., Schlosser, M. T. & Garcia-Molina, H. The EigenTrust algorithm for reputation management in P2P networks. in *Proc. 12th Int. Conf. World Wide Web (WWW)* 640–651 (ACM, 2003).

[20] Jøsang, A. & Ismail, R. The beta reputation system. in *Proc. 15th Bled Electronic Commerce Conf.* paper 41 (AIS eLibrary, 2002). https://aisel.aisnet.org/bled2002/41.

[21] Chen, I.-R., Bao, F. & Guo, J. Trust-based service management for Social Internet of Things systems. *IEEE Trans. Dependable Secure Comput.* **13**, 684–696 (2016).

[22] Yan, Z., Zhang, P. & Vasilakos, A. V. A survey on trust management for Internet of Things. *J. Netw. Comput. Appl.* **42**, 120–134 (2014).

[23] Guo, J., Chen, I.-R. & Tsai, J. J. P. A survey of trust computation models for service management in Internet of Things systems. *Comput. Commun.* **97**, 1–14 (2017).

[24] Konsta, A. M., Lluch Lafuente, A. & Dragoni, N. A survey of trust management for Internet of Things. *IEEE Access* **11**, 122175–122204 (2023).

[25] Hussain, Y. *et al.* Context-aware trust and reputation model for fog-based IoT. *IEEE Access* **8**, 31622–31632 (2020).

[26] Bonomi, F., Milito, R., Zhu, J. & Addepalli, S. Fog computing and its role in the Internet of Things. in *Proc. First MCC Workshop on Mobile Cloud Computing* 13–16 (ACM, 2012).

[27] Chiang, M. & Zhang, T. Fog and IoT: an overview of research opportunities. *IEEE Internet Things J.* **3**, 854–864 (2016).

[28] Shi, W., Cao, J., Zhang, Q., Li, Y. & Xu, L. Edge computing: vision and challenges. *IEEE Internet Things J.* **3**, 637–646 (2016).

[29] Debe, M., Salah, K., Rehman, M. H. U. & Svetinovic, D. IoT public fog nodes reputation system: a decentralized solution using Ethereum blockchain. *IEEE Access* **7**, 178082–178093 (2019).

[30] Haber, S. & Stornetta, W. S. How to time-stamp a digital document. *J. Cryptol.* **3**, 99–111 (1991).

[31] Schneier, B. & Kelsey, J. Secure audit logs to support computer forensics. *ACM Trans. Inf. Syst. Secur.* **2**, 159–176 (1999).

[32] Crosby, S. A. & Wallach, D. S. Efficient data structures for tamper-evident logging. in *Proc. 18th USENIX Security Symposium* 317–334 (USENIX Association, 2009).

[33] Laurie, B., Langley, A. & Kasper, E. Certificate Transparency. RFC 6962 (RFC Editor, 2013). https://doi.org/10.17487/RFC6962.

[34] Buldas, A., Kroonmaa, A. & Laanoja, R. Keyless signatures' infrastructure: how to build global distributed hash-trees. in *Secure IT Systems (NordSec 2013)*, Lecture Notes in Computer Science Vol. 8208, 313–320 (Springer, 2013).

[35] Kalodner, H., Goldfeder, S., Chen, X., Weinberg, S. M. & Felten, E. W. Arbitrum: scalable, private smart contracts. in *Proc. 27th USENIX Security Symposium* 1353–1370 (USENIX Association, 2018).

[36] Meidan, Y. *et al.* N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Comput.* **17**, 12–22 (2018).

[37] Douceur, J. R. The Sybil attack. in *Peer-to-Peer Systems (IPTPS 2002)*, Lecture Notes in Computer Science Vol. 2429, 251–260 (Springer, 2002).

[38] Friedman, E. J. & Resnick, P. The social cost of cheap pseudonyms. *J. Econ. Manag. Strategy* **10**, 173–199 (2001).

[39] Malkhi, D. & Reiter, M. K. Byzantine quorum systems. *Distrib. Comput.* **11**, 203–213 (1998).

[40] Castro, M. & Liskov, B. Practical Byzantine fault tolerance and proactive recovery. *ACM Trans. Comput. Syst.* **20**, 398–461 (2002).

[41] Gupta, H., Dastjerdi, A. V., Ghosh, S. K. & Buyya, R. iFogSim: a toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments. *Softw. Pract. Exp.* **47**, 1275–1296 (2017).

[42] Bodik, P. *et al.* Intel Lab data. Intel Berkeley Research Lab https://db.csail.mit.edu/labdata/labdata.html (2004; accessed 12 October 2026).

[43] Jacob, R., Da Forno, R., Trüb, R., Biri, A. & Thiele, L. Dataset: wireless link quality estimation on FlockLab – and beyond. in *Proc. 2nd Workshop on Data Acquisition To Analysis (DATA)* 57–60 (ACM, 2019).

[44] Lim, R. *et al.* FlockLab: a testbed for distributed, synchronized tracing and profiling of wireless embedded systems. in *Proc. 12th Int. Conf. Information Processing in Sensor Networks (IPSN)* 153–166 (ACM, 2013).

[45] Amazon Web Services. Availability Zones. in *AWS Fault Isolation Boundaries* (AWS whitepaper) https://docs.aws.amazon.com/whitepapers/latest/aws-fault-isolation-boundaries/availability-zones.html (accessed 12 October 2026).

[46] Page, E. S. Continuous inspection schemes. *Biometrika* **41**, 100–115 (1954).

Author contributions. A.A. conceived the study, designed the extensions, implemented the simulator, performed the experiments and analysis, and wrote the manuscript.

Funding. This work was supported by Prince Sattam bin Abdulaziz University under Project PSAU/2026/R/1448.

Additional information. Competing interests: The author declares no competing interests.